Facial Keypoints Detection using Convolutional Neural Networks

> ***Abstract:*** *This report outlines the development, architecture, and training of a Convolutional Neural Network (CNN) designed for facial keypoint detection. Built using PyTorch, the network processes 96×96 grayscale images to predict the coordinates of 15 facial landmarks. Through progressive spatial downsampling and channel expansion, the model achieved a final Mean Squared Error (MSE) loss of 0.0016 after 15 training epochs.*

1. Introduction

Facial landmark detection is a fundamental computer vision task that involves identifying the spatial coordinates of key facial features such as the eyes, nose, and mouth corners. This report documents a deep learning methodology to automate this extraction using a custom Convolutional Neural Network (CNN) architecture implemented in PyTorch.

2. Methodology

2.1 Data Preprocessing

The dataset, formatted for a Kaggle competition, comprises training and test sets provided in CSV format. The preprocessing workflow involves:

- **Image Parsing:** Space-separated string values are converted into 96×96 NumPy arrays.
- **Image Normalization:** Pixel values are scaled from [0, 255] to a [0, 1] range to facilitate stable gradient descent.
- **Keypoint Normalization:** Target coordinates are shifted and scaled from the [0, 96] dimension to a [-1, 1] interval. For training stability, samples with missing keypoint annotations are dropped.

2.2 Network Architecture

The model architecture applies a sequence of four convolutional layers to extract hierarchical features, culminating in two fully connected linear layers. Max pooling reduces spatial resolution while increasing the receptive field.

Layer	Input Size	Operations	Output Size
Conv Block 1	1 × 96 × 96	Conv2d (32 filters, 3x3, pad 1) → ReLU → MaxPool (2x2)	32 × 48 × 48

Conv Block 2	32 × 48 × 48	Conv2d (64 filters, 3x3, pad 1) → ReLU → MaxPool (2x2)	64 × 24 × 24
Conv Block 3	64 × 24 × 24	Conv2d (128 filters, 3x3, pad 1) → ReLU → MaxPool (2x2)	128 × 12 × 12
Conv Block 4	128 × 12 × 12	Conv2d (256 filters, 3x3, pad 1) → ReLU → MaxPool (2x2)	256 × 6 × 6
Flatten	256 × 6 × 6	View / Reshape	9216
FC 1	9216	Linear (512 units) → ReLU	512
FC 2	512	Linear (30 units)	30

The final layer outputs a vector of size 30, corresponding to the (x, y) coordinates for all 15 keypoints.

3. Training and Results

The CNN was trained using PyTorch with the following hyperparameter configuration:

- **Optimizer:** Adam Optimizer
- **Learning Rate:** 0.001
- **Batch Size:** 64
- **Loss Function:** Mean Squared Error (MSE)

The model was trained over 15 epochs. The training process demonstrated excellent convergence properties:

- **Epoch 1:** Initial loss of 0.0762
- **Epoch 2:** Sharp drop to 0.0052
- **Epoch 10:** Consistent improvement to 0.0029
- **Epoch 15:** Final converged loss of 0.0016

The trained parameters (weights and biases) are preserved in the serialized `data.pkl / model.pth` files.

4. Conclusion and Inference

Inference on the test set (`test.csv`) is performed with gradient calculation disabled. The network's [-1, 1] output is unnormalized back to the 96×96 pixel scale. Because minor network inaccuracies might predict coordinates outside the image frame, all output values are clipped strictly to the [0, 96] boundary.

The wide-format predictions are subsequently melted into a long format. By merging with `IdLookupTable.csv` based on `ImageId` and `FeatureName`, the final `submission.csv` file is generated, matching the exact row IDs required for evaluation.